

VIVA Preparation Guide

Project Topics, Code Working, and Exam-Ready Questions

Based on the serial-mapped PMAF code collection

Typeface: JetBrains Mono

Contents

- 1.Lab 1: Exploratory Data Analysis on Sales Data
- 2.Lab 2: Data Cleaning and Preprocessing on Diabetes Data
- 3.Lab 3: Linear Regression on Cereal Calories
- 4.Lab 4: Logistic Regression for Marketing Purchase Prediction
- 5.Lab 5A: Decision Tree Classification on Diabetes Data
- 6.Lab 5B: Decision Tree Classification on Heart Disease Data
- 7.Lab 6A: K-Means Clustering on Iris Dataset
- 8.Lab 6B: Hierarchical Clustering on Loan Data
- 9.Lab 7: Association Rule Mining on Grocery Transactions
- 10.Extra 8: Time-Series Forecasting using ARIMA
- 11.Extra 9: Machine Learning Forecasting and Explainability
- 12.Extra 10: Deep Learning Forecasting and Flask Deployment

Project File Map

Serial	Topic	Code File	Main Method
Lab 1	Exploratory Data Analysis on Sales Data	01_eda_sales.py	EDA
Lab 2	Data Cleaning and	02_dcp_diabetes.py	DCP

Serial	Topic	Code File	Main Method
	Preprocessing on Diabetes Data		
Lab 3	Linear Regression on Cereal Calories	03_linreg_cereal.py	LINREG
Lab 4	Logistic Regression for Marketing Purchase Prediction	04_logreg_marketing.py	LOGREG
Lab 5A	Decision Tree Classification on Diabetes Data	05_dt_diabetes.py	DT
Lab 5B	Decision Tree Classification on Heart Disease Data	05_dt_heart_disease.py	DT
Lab 6A	K-Means Clustering on Iris Dataset	06_kmeans_iris.py	KMEANS
Lab 6B	Hierarchical Clustering on Loan Data	06_hc_loan.py	HC
Lab 7	Association Rule Mining on Grocery Transactions	07_arm_grocery.py	ARM
Extra 8	Time-Series Forecasting using ARIMA	08_arima_air_passengers.py	ARIMA
Extra 9	Machine Learning Forecasting and Explainability	09_ml_forecasting_explainability.py	ML
Extra 10	Deep Learning Forecasting and Flask Deployment	10_lstm_stock_deployment.py	LSTM

Lab 1: Exploratory Data Analysis on Sales Data

File: `01_eda_sales.py` Dataset: `sales_data.csv` Abbreviation: EDA

Concept

Exploratory Data Analysis is the first practical step in understanding a dataset before modelling.

In this project, sales records are inspected through summary statistics, missing-value checks, duplicate removal, correlation analysis, histograms, boxplots, scatter plots, monthly trends, and category-wise sales grouping.

The goal is not prediction yet. The goal is to discover patterns, quality issues, outliers, and business signals that guide later modelling decisions.

How the Code Works

- 1.Imports pandas, numpy, matplotlib, and seaborn for data handling and visualisation.
- 2.Loads sales_data.csv; if the file is missing, the script creates sample sales data so the program remains runnable.
- 3.Displays first rows, dataset information, missing values, and descriptive statistics.
- 4.Drops duplicate rows to avoid repeated records affecting analysis.
- 5.Plots a numeric correlation heatmap to identify related variables.
- 6.Creates histogram, boxplot, and scatter plots for sales behaviour.
- 7.Groups monthly and categorical sales values to compare performance across time and business categories.

Viva Questions

| Q: What is EDA?

| A: EDA is the process of analysing and visualising a dataset to understand its structure, patterns, missing values, outliers, and relationships before applying machine learning.

| Q: Why do we check missing values?

| A: Missing values can distort statistics and model training. Checking them early helps decide whether to remove, replace, or impute them.

| Q: Why is a correlation heatmap used?

| A: A correlation heatmap shows the strength and direction of relationships between numeric variables, helping identify useful features and multicollinearity.

| Q: What is the use of a boxplot?

| A: A boxplot summarises distribution using quartiles and highlights possible outliers.

| Q: Why group sales by month or category?

| A: Grouping reveals trends, seasonality, and which categories or regions contribute most to sales.

Lab 2: Data Cleaning and Preprocessing on Diabetes Data

File: 02_dcp_diabetes.py Dataset: diabetes.csv Abbreviation: DCP

Concept

Data preprocessing prepares raw data for machine learning by handling invalid values, missing values, duplicates, and scale differences.

The diabetes dataset contains medical measurements such as glucose, blood pressure, BMI, insulin, and outcome.

Some medical columns cannot realistically be zero, so zero values are treated as invalid and replaced before scaling.

How the Code Works

1. Loads diabetes.csv or creates a small sample dataframe.
2. Displays first rows, data information, missing-value count, and invalid zero values.
3. Replaces zero values in glucose, blood pressure, skin thickness, insulin, and BMI with NaN.

4. Fills missing values using the column mean.
5. Drops duplicate records.
6. Separates features from the target Outcome column.
7. Applies StandardScaler so all features have comparable scale.
8. Saves the cleaned output as cleaned_diabetes.csv.

Viva Questions

| Q: What is preprocessing?

| A: Preprocessing is the transformation of raw data into a clean and suitable format for analysis or model training.

| Q: Why are zero values replaced in this dataset?

| A: For columns like glucose, blood pressure, and BMI, zero is medically invalid, so it is treated as missing data.

| Q: What is StandardScaler?

| A: StandardScaler converts features to zero mean and unit variance, preventing large-scale features from dominating models.

| Q: Why separate X and y?

| A: X contains independent input features, while y contains the target variable the model or analysis is based on.

| Q: Why save cleaned_diabetes.csv?

| A: Saving the cleaned file allows later assignments to reuse preprocessed data without repeating all cleaning steps.

Lab 3: Linear Regression on Cereal Calories

File: 03_linreg_cereal.py Dataset: cereal.csv Abbreviation: LINREG

Concept

Linear Regression is a supervised learning algorithm used to predict a continuous

numeric target.

Here, cereal calories are predicted from nutritional features such as protein, fat, sodium, fibre, carbohydrates, sugar, and potassium.

The model learns coefficients that estimate how each feature contributes to the target value.

How the Code Works

1. Loads cereal.csv or creates sample cereal nutrition data.
2. Selects numeric nutrition columns as features and calories as the target.
3. Fills missing numeric values using the mean.
4. Splits data into training and testing sets.
5. Creates and trains a LinearRegression model.
6. Predicts calories for the test set.
7. Evaluates the model using MAE, MSE, RMSE, and R2 score.
8. Prints feature coefficients and intercept for interpretation.

Viva Questions

| **Q: What is Linear Regression?**

| A: Linear Regression predicts a continuous value by fitting a linear relationship between input features and the target.

| **Q: What does a coefficient mean?**

| A: A coefficient shows how much the prediction changes when that feature increases by one unit, assuming other features are fixed.

| **Q: What is R2 score?**

| A: R2 score measures how much variance in the target is explained by the model. Higher values usually indicate better fit.

| **Q: Why split train and test data?**

| A: The split checks whether the model generalises to unseen data instead of only memorising training data.

| **Q: What is RMSE?**

| A: Root Mean Squared Error is the square root of MSE and represents average
| prediction error in the target's unit.

Lab 4: Logistic Regression for Marketing Purchase Prediction

File: 04_logreg_marketing.py Dataset: Social_Network_Ads.csv Abbreviation:

LOGREG

Concept

Logistic Regression is a supervised classification algorithm used when the output is categorical, often binary.

In this project, the model predicts whether a user purchased a product based on age and estimated salary.

Unlike Linear Regression, Logistic Regression maps output through a sigmoid function to produce class probabilities.

How the Code Works

1. Loads Social_Network_Ads.csv or creates sample marketing data.
2. Selects Age and EstimatedSalary as input features.
3. Uses Purchased as the target class.
4. Splits the dataset into training and testing data.
5. Scales features using StandardScaler.
6. Trains a LogisticRegression classifier.
7. Predicts test labels.
8. Prints accuracy, confusion matrix, and classification report.

Viva Questions

| Q: Why is Logistic Regression used here?

| A: It is used because Purchased is a binary class, so the task is
| classification, not continuous prediction.

| Q: What is a confusion matrix?

| A: A confusion matrix compares actual and predicted classes using true positives, true negatives, false positives, and false negatives.

| Q: Why scale Age and Salary?

| A: Salary values are much larger than age values, so scaling prevents one feature from dominating model training.

| Q: What is accuracy?

| A: Accuracy is the ratio of correct predictions to total predictions.

| Q: What is the sigmoid function?

| A: The sigmoid function maps any real value to a probability between 0 and 1.

Lab 5A: Decision Tree Classification on Diabetes Data

File: 05_dt_diabetes.py Dataset: diabetes.csv Abbreviation: DT

Concept

Decision Tree is a supervised machine learning algorithm that makes predictions using a tree of if-else decisions.

For diabetes classification, the model predicts Outcome using medical features.

The tree chooses splits that separate classes effectively using criteria such as entropy or gini impurity.

How the Code Works

1. Loads diabetes.csv or creates sample diabetes data.
2. Replaces invalid medical zero values with median values.
3. Splits features and target Outcome.
4. Splits data into training and testing sets.
5. Creates a DecisionTreeClassifier with entropy and max_depth.
6. Trains the tree and predicts test labels.
7. Displays accuracy, confusion matrix, classification report, and the tree plot.

Viva Questions

| Q: What is a Decision Tree?

| A: A Decision Tree is a model that predicts by splitting data based on feature conditions until reaching a class decision.

| Q: What is entropy?

| A: Entropy measures impurity or uncertainty in a dataset. Lower entropy means purer class separation.

| Q: Why use max_depth?

| A: max_depth limits tree growth and reduces overfitting.

| Q: What is overfitting in trees?

| A: Overfitting happens when the tree becomes too complex and learns noise instead of general patterns.

| Q: Why visualise the tree?

| A: Tree visualisation helps explain which features and thresholds drive classification.

Lab 5B: Decision Tree Classification on Heart Disease Data

File: 05_dt_heart_disease.py Dataset: heart_disease.csv Abbreviation: DT
--

Concept

This assignment applies the same Decision Tree idea to heart disease classification.

The target variable indicates disease presence or absence, while health measurements become predictor features.

The project demonstrates how a classification workflow can be reused across different medical datasets.

How the Code Works

1. Loads `heart_disease.csv` or creates a sample heart disease dataframe.
2. Checks missing values and removes incomplete records.
3. Converts categorical columns using one-hot encoding if required.
4. Splits data into features and target.
5. Trains a `DecisionTreeClassifier` using entropy and controlled depth.
6. Evaluates using accuracy, confusion matrix, and classification report.
7. Plots the final decision tree for explanation.

Viva Questions

| Q: Why use `get_dummies`?

| A: `get_dummies` converts categorical values into numeric columns so machine learning algorithms can process them.

| Q: What is the target column?

| A: The target column is `target`, representing whether heart disease is present or absent.

| Q: How is this different from diabetes classification?

| A: The algorithm is similar, but the dataset features and medical interpretation are different.

| Q: What does `classification_report` show?

| A: It shows precision, recall, f1-score, and support for each class.

| Q: Why is interpretability important in medical models?

| A: Medical predictions need explanations so users can understand which features influenced a decision.

Lab 6A: K-Means Clustering on Iris Dataset

File: <code>06_kmeans_iris.py</code> Dataset: <code>Iris.csv</code> / <code>sklearn iris</code> Abbreviation: KMEANS

Concept

K-Means is an unsupervised learning algorithm used to group similar records into clusters.

The Iris dataset contains flower measurements such as sepal and petal dimensions.

The model attempts to form three clusters, which can be compared with the three natural Iris species.

How the Code Works

1. Loads the Iris dataset.
2. Creates a dataframe from flower measurements.
3. Selects numeric features for clustering.
4. Initialises KMeans with three clusters.
5. Fits the model and predicts cluster labels.
6. Adds cluster labels back into the dataframe.
7. Plots clusters using the first two features.

Viva Questions

| Q: What is unsupervised learning?

| A: Unsupervised learning finds patterns in data without labelled target values.

| Q: What does K mean in K-Means?

| A: K is the number of clusters the algorithm should form.

| Q: How does K-Means work?

| A: It places centroids, assigns points to nearest centroids, updates centroids,
| and repeats until stable.

| Q: Why use three clusters for Iris?

| A: The Iris dataset has three known species, so three clusters are a meaningful
| choice.

| Q: What is a centroid?

| A: A centroid is the central point representing a cluster.

Lab 6B: Hierarchical Clustering on Loan Data

File: `06_hc_loan.py` Dataset: `loan.csv` Abbreviation: HC

Concept

Hierarchical Clustering builds a hierarchy of clusters instead of directly selecting flat groups at the beginning.

Agglomerative clustering starts with each record as a separate cluster and repeatedly merges the closest clusters.

A dendrogram visualises how clusters merge at different distance levels.

How the Code Works

1. Loads `loan.csv`.
2. Selects numeric columns for clustering.
3. Fills missing numeric values using mean values.
4. Scales features with `StandardScaler`.
5. Computes linkage using Ward's method.
6. Plots a dendrogram to show cluster hierarchy.
7. Applies `AgglomerativeClustering` with three clusters.
8. Adds cluster labels to the loan dataframe and plots cluster separation.

Viva Questions

| Q: What is hierarchical clustering?

| A: It is an unsupervised method that creates nested clusters based on similarity.

| Q: What is a dendrogram?

| A: A dendrogram is a tree-like diagram showing how clusters are merged or split.

| Q: What is Ward linkage?

| A: Ward linkage merges clusters in a way that minimises the increase in within-cluster variance.

| Q: Why scale loan data?

| A: Scaling ensures variables with large numeric ranges do not dominate distance calculations.

| Q: Difference between K-Means and hierarchical clustering?

| A: K-Means partitions data around centroids, while hierarchical clustering builds a tree of cluster relationships.

Lab 7: Association Rule Mining on Grocery Transactions

File: `07_arm_grocery.py` Dataset: `grocery_dataset.txt` Abbreviation: **ARM**

Concept

Association Rule Mining discovers relationships between items that frequently appear together in transactions.

Market basket analysis uses this technique to find rules like customers who buy bread also buy butter.

Apriori is used to generate frequent itemsets and `association_rules` converts them into interpretable rules.

How the Code Works

1. Reads `grocery_dataset.txt` transaction by transaction.
2. Builds a set of all unique grocery items.
3. Converts transactions into a one-hot encoded dataframe.
4. Counts item frequency and plots the top items.
5. Runs `apriori` with a minimum support threshold.
6. Generates rules using lift as the metric.
7. Displays the strongest association rules.

Viva Questions

| Q: What is support?

| A: Support is the fraction of transactions containing an itemset.

| Q: What is confidence?

| A: Confidence measures how often the consequent appears when the antecedent appears.

| Q: What is lift?

| A: Lift compares rule confidence with the expected occurrence if items were independent. Lift greater than 1 indicates positive association.

| Q: Why one-hot encode transactions?

| A: Apriori needs data in a binary item-present/item-absent format.

| Q: Where is association mining used?

| A: It is used in market basket analysis, recommendation systems, and product placement decisions.

Extra 8: Time-Series Forecasting using ARIMA

File: 08_arima_air_passengers.py Dataset: AirPassengers.csv / sample data

Abbreviation: ARIMA

Concept

Time-series forecasting predicts future values from historical ordered observations.

ARIMA combines autoregression, differencing, and moving averages to model temporal patterns.

The project forecasts airline passengers for future months.

How the Code Works

1. Loads AirPassengers.csv or creates sample passenger data.
2. Converts Month to datetime and sets it as the index.
3. Plots the original time series.
4. Creates an ARIMA model with order (2, 1, 2).
5. Fits the model to historical passenger values.
6. Forecasts the next 12 months.
7. Plots original and forecasted values.

Viva Questions

| Q: What is a time series?

| A: A time series is data collected in chronological order, such as monthly sales or passenger counts.

| Q: What does ARIMA stand for?

| A: ARIMA stands for AutoRegressive Integrated Moving Average.

| Q: What does differencing do?

| A: Differencing removes trend and helps make a time series stationary.

| Q: Why set date as index?

| A: Time-series models and plots use the date index to preserve chronological order.

| Q: What is forecasting?

| A: Forecasting estimates future values based on past patterns.

Extra 9: Machine Learning Forecasting and Explainability

File: 09_ml_forecasting_explainability.py Dataset: sales.csv / sample data

Abbreviation: ML

Concept

This project compares machine learning models for forecasting and adds explainability through feature importance.

It uses models such as Linear Regression, Random Forest, and Gradient Boosting to predict sales.

Explainability helps identify which input features contribute most to predictions.

How the Code Works

1. Loads sales.csv or sample sales data.
2. Prepares date-based and numeric features.
3. Splits the dataset into train and test sets.
4. Trains multiple regression models.
5. Evaluates model performance using R2 or error metrics.
6. Compares model scores visually.
7. Prints or plots feature importance for interpretable models.

Viva Questions

| Q: Why compare multiple models?

| A: Different models capture different patterns, so comparison helps select the most suitable one.

| Q: What is explainability?

| A: Explainability means understanding why a model made a prediction or which features influenced it.

| Q: What is feature importance?

| A: Feature importance ranks input variables based on their contribution to prediction.

| Q: Why use Random Forest?

| A: Random Forest reduces variance by combining many decision trees.

| Q: What is model evaluation?

| A: Model evaluation measures how well predictions match actual values on unseen
| data.

Extra 10: Deep Learning Forecasting and Flask Deployment

File: <code>10_lstm_stock_deployment.py</code> Dataset: <code>stock_data.csv</code> / sample data

Abbreviation: LSTM

Concept

LSTM is a recurrent neural network architecture designed to learn sequential dependencies.

It is useful for stock or time-series forecasting because it can retain information over previous time steps.

The project also includes a Flask interface so the trained prediction logic can be served as a small web application.

How the Code Works

1. Loads stock price data or creates sample data.
2. Scales the price values before neural-network training.
3. Creates time-window sequences for supervised learning.
4. Builds an LSTM model with dense output layers.
5. Trains the model and saves it as `lstm_model.h5`.
6. Plots predicted vs actual stock prices.
7. Creates a Flask app route to display prediction output.

Viva Questions

| Q: What is LSTM?

| A: LSTM stands for Long Short-Term Memory, a neural network designed for
| sequence learning.

| Q: Why scale data for LSTM?

| A: Scaling improves neural-network training stability and convergence.

| Q: What is a time window?

| A: A time window is a fixed number of previous observations used to predict the next value.

| Q: Why use Flask?

| A: Flask is used to deploy the model through a simple web interface.

| Q: Why was TensorFlow not installed in this venv?

| A: The current Pythonproject venv uses Python 3.14, and pip did not provide a compatible TensorFlow build for that version.